Practical 14

Part A:

Aim:

1. Assemble and test: build a tiny layered app (a model, a service using an ArrayList, and a menu main), then write two or three JUnit @Test methods checking that adding increases the count and an invalid action throws — practice before testing MiniBank.

Code:

Task.java

```
public class Task {
    private final int id;
    private final String description;

    public Task(int id, String description) {
        if (id <= 0) {
            throw new IllegalArgumentException("Task ID must be
positive");
        }

        if (description == null || description.isBlank()) {
            throw new IllegalArgumentException(
                    "Task description cannot be empty");
        }

        this.id = id;
        this.description = description;
    }

    public int getId() {
```

```
        return id;
    }

    public String getDescription() {
        return description;
    }

    @Override
    public String toString() {
        return id + ". " + description;
    }
}
```

TaskService.java

```
import java.util.ArrayList;
import java.util.List;

public class TaskService {
    private final List<Task> tasks = new ArrayList<>();
    private int nextId = 1;

    public Task addTask(String description) {
        Task task = new Task(nextId++, description);
        tasks.add(task);
        return task;
    }

    public int getTaskCount() {
        return tasks.size();
    }

    public List<Task> getAllTasks() {
        return new ArrayList<>(tasks);
    }

    public void removeTask(int id) {
        boolean removed = tasks.removeIf(task -> task.getId()
== id);

        if (!removed) {
            throw new IllegalArgumentException(
                    "Task ID not found: " + id);
        }
    }
}
```

TaskApp.java

```
import java.util.Scanner;

public class TaskApp {
    public static void main(String[] args) {
        TaskService service = new TaskService();

        try (Scanner scanner = new Scanner(System.in)) {
            boolean running = true;

            while (running) {
                System.out.println("\n--- Task Manager ---");
                System.out.println("1. Add task");
                System.out.println("2. View tasks");
                System.out.println("3. Remove task");
                System.out.println("4. Exit");
                System.out.print("Choose an option: ");

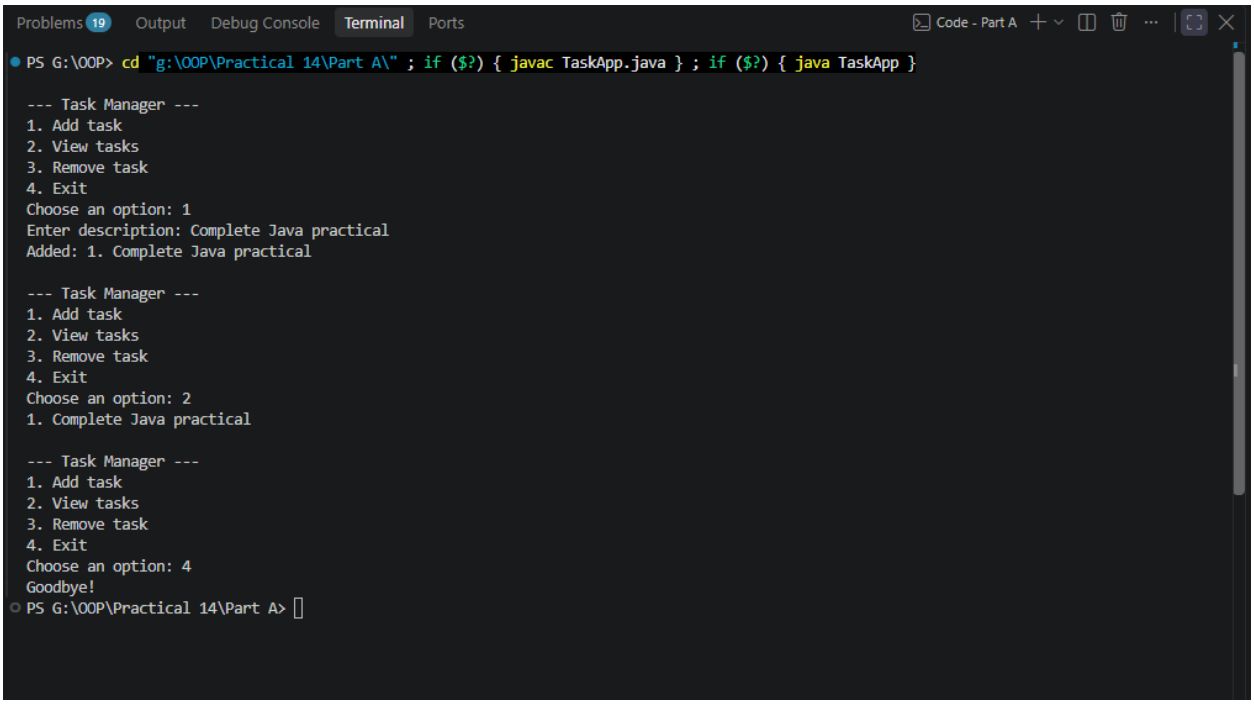
                String input = scanner.nextLine();

                try {
                    switch (input) {
                        case "1" -> {
                            System.out.print("Enter
description: ");
                            Task task =
service.addTask(scanner.nextLine());
                            System.out.println("Added: " +
task);
                        }
                        case "2" -> {
                            if
(service.getAllTasks().isEmpty()) {
```

```
                                System.out.println("No tasks
available.");
                            } else {
                                service.getAllTasks()
                                        .forEach(System.out::pr
intln);
                            }
                        }
                        case "3" -> {
                            System.out.print("Enter task ID:
");
                            int id =
Integer.parseInt(scanner.nextLine());
                            service.removeTask(id);
                            System.out.println("Task
removed.");
                        }
                        case "4" -> running = false;
                        default -> System.out.println("Invalid
menu option.");
                    }
                } catch (IllegalArgumentException e) {
                    System.out.println("Error: " +
e.getMessage());
                }
            }
        }

        System.out.println("Goodbye!");
    }
}
```

Output:


Problems 19
Output
Debug Console
Terminal
Ports
Code - Part A
PS G:\OOP> cd "g:\OOP\Practical 14\Part A\" ; if ($?) { javac TaskApp.java } ; if ($?) { java TaskApp }
--- Task Manager ---
1. Add task
2. View tasks
3. Remove task
4. Exit
Choose an option: 1
Enter description: Complete Java practical
Added: 1. Complete Java practical
--- Task Manager ---
1. Add task
2. View tasks
3. Remove task
4. Exit
Choose an option: 2
1. Complete Java practical
--- Task Manager ---
1. Add task
2. View tasks
3. Remove task
4. Exit
Choose an option: 4
Goodbye!
PS G:\OOP\Practical 14\Part A>

2. (For Advanced Learners – home) Organise that app into packages, build a runnable JAR, and add a test that saves and reloads its data unchanged.

Code:

Pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>lab.tasks</groupId>
    <artifactId>task-manager</artifactId>
    <version>1.0.0</version>

    <properties>
        <maven.compiler.release>21</maven.compiler.release>
        <project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding>
        <junit.version>5.11.4</junit.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>${junit.version}</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
```

```
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.5.2</version>
            </plugin>

            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-jar-plugin</artifactId>
                <version>3.4.2</version>
                <configuration>
                    <archive>
                        <manifest>
                            <mainClass>lab.tasks.TaskApp</mainC
lass>
                        </manifest>
                    </archive>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

Key Questions / Analysis / Interpretation to be evaluated during/after Implementation

1. What is a layered architecture (model, service, user interface), and why is it useful?

Ans.

Layered architecture organizes an application into separate layers, each with a specific responsibility.

Layer	Responsibility	Example in Task Manager
Model	Represents and stores data	`Task.java`
Service	Implements business logic and manages operations	`TaskService.java`
User interface	Accepts user input and displays results	`TaskApp.java`

Why is it useful?

- Separation of concerns: Each layer has a clear responsibility.
- Maintainability: Changes to one layer usually require fewer changes to others.
- Testability: The service can be tested independently of the user interface.
- Reusability: The same service can be used by a console application, a web application, or another interface.

2. How do you assert in JUnit that a method throws an expected exception?

Ans.

Use JUnit 5's `assertThrows()` method. It verifies that the specified exception type is thrown when the tested operation executes.

Syntax:

```
assertThrows(
    IllegalArgumentException.class,
    () -> service.removeTask(999)
);
```

Here, the test passes if `removeTask(999)` throws an `IllegalArgumentException`, as expected when task ID `999` does not exist.

3. What does packaging the application as a runnable JAR provide?

Ans.

A runnable JAR packages compiled Java classes and application resources into a single Java Archive file. Its manifest specifies the main class that should execute when the JAR is launched.

Advantages:

- Portability: The application can be distributed as one archive.
- Easy execution: Run it using `java -jar filename.jar`.
- Simplified deployment: Users do not need to compile the source code again.
- Organization: Compiled classes and resources are packaged together.